Safenode

Safenode Threat Model and Risk Register

Threats, mitigations, residual risks, and release-blocking security decisions.

Document version: 2026-07-27
Product: Safenode
Classification: Internal / customer-safe where noted

Scope and assumptions

This model covers the web and native client, API, database, identity flows, billing/email integrations, deployment pipeline, and team collaboration. It assumes TLS is correctly configured, the user device is not fully compromised, and deployment credentials are protected.

Primary threats

Threat	Impact	Mitigation	Residual risk
XSS or malicious extension	Vault session compromise	CSP/security headers, memory-only raw key, no persistent master password	Active browser compromise can read unlocked memory
Phishing/passkey prompt abuse	Account takeover	WebAuthn/passkeys, origin binding, user education	Users can approve a malicious prompt
Lost sole device	Account lockout or data loss	Recovery kit, device re-approval, successor workflow	Recovery material can be lost or exposed
Malicious backend operator	Vault disclosure	Zero-knowledge ciphertext design, no server decryption path	Metadata and availability remain exposed
Webhook forgery	Incorrect entitlements	Paddle signature verification and idempotency	Provider/account misconfiguration
Leaked deployment secret	Service compromise	Rotation runbook, secret manager, no env dumps in repo	Unrotated historical credentials remain dangerous
Supply-chain dependency issue	Build/runtime compromise	Lockfiles, CI, audit checks, minimal production dependencies	Transitive vulnerabilities require monitoring

Security controls

- Authentication: WebAuthn/passkeys, email verification, session cookies/tokens, device binding.
- Authorization: ownership checks, team membership checks, subscription entitlements, device policy.
- Cryptography: WebCrypto, AES-GCM, Argon2id, HKDF, wrapped vault keys, recovery-kit crypto.
- Application defense: Zod validation, helmet/security headers, CORS policy, rate limits, audit logs.
- Operations: CI gates, Prisma schema sync, Sentry, signed releases, association files, branch protection.

Residual risks requiring explicit acceptance

- A compromised unlocked endpoint can access decrypted vault content; zero-knowledge does not protect an active endpoint.
- PRF support varies by browser and authenticator; fallback unlock remains necessary.
- Email delivery and account recovery depend on external provider availability and user control of the mailbox.
- The current subscription record persists period dates and price ID; billing cycle is inferred from the configured price ID rather than a dedicated field.
- Native release signing, notarization, and store distribution require platform credentials and manual verification.

Release-blocking findings

Do not ship a production release if any of the following is true:

- A production secret appears in Git history or deployment logs and has not been rotated.
- Paddle webhook signatures are not verified or idempotency is disabled.
- The client persists a raw vault key, master password, recovery secret, or PRF output.
- The production API, email sending domain, or iOS/Android association files are unavailable.
- CI security checks or backend/frontend tests are failing.